

SOLID

Examples

Vladimir Soroka

The Customer is always right! Or is he?



```
class Customer {
private:
    string name;
    int id;
    vector<Item> items;
    float totalAmount;
public:
    void setName(string name);
    string getName();
    void setId(int id);
    int getId();
    void addItem(Item item);
    void removeItem(Item item);
    float calculateTotalAmount();
    string generateInvoice();
};
```

That's the way to do it!

```
class CustomerDetails {
private:
    string name;
    int id;
public:
    void setName(string name);
    string getName();
    void setId(int id);
    int getId();
};

class BillingCalculator {
private:
    vector<Item> items;
public:
    void addItem(Item item);
    void removeItem(Item item);
    float calculateTotalAmount();
};

class InvoiceGenerator {
public:
    string generateInvoice(CustomerDetails customerDetails, BillingCalculator bill
};
```

Let's get in shape!

```
enum class ShapeType { Circle, Rectangle };

class Shape {
public:
    ShapeType type;
    double radius;
    double width, height;
};

class AreaCalculator {
public:
    double calculateArea(const Shape& shape) {
        if (shape.type == ShapeType::Circle) {
            return 3.14 * shape.radius * shape.radius;
        } else if (shape.type == ShapeType::Rectangle) {
            return shape.width * shape.height;
        }
        return 0;
    }
};
```

That's the way to do it!

```
class Shape {
public:
    virtual double area() const = 0;
    virtual ~Shape() = default;
};

class Circle : public Shape {
    double radius;
public:
    Circle(double r) : radius(r) {}
    double area() const override {
        return 3.14 * radius * radius;
    }
};

class Rectangle : public Shape {
    double width, height;
public:
    Rectangle(double w, double h) : width(w), height(h) {}
    double area() const override {
        return width * height;
    }
};

class AreaCalculator {
public:
    double calculateArea(const Shape& shape) {
        return shape.area(); // No need to modify when adding new shapes
    }
};
```



To write or not to write? That is the question!

```
class File {
public:
    virtual void read() = 0;
    virtual void write(const std::string& data) = 0;
    virtual ~File() = default;
};

class ReadOnlyFile : public File {
public:
    void read() override {
        std::cout << "Reading from read-only file\n";
    }

    void write(const std::string& data) override {
        throw std::runtime_error("Cannot write to a read-only file");
    }
};
```

That's the way to do it!

```
class IReadable {
public:
    virtual void read() = 0;
    virtual ~IReadable() = default;
};

class IWritable {
public:
    virtual void write(const std::string& data) = 0;
    virtual ~IWritable() = default;
};

class ReadOnlyFile : public IReadable {
public:
    void read() override {
        std::cout << "Reading from read-only file\n";
    }
};

class ReadWriteFile : public IReadable, public IWritable {
public:
    void read() override {
        std::cout << "Reading from read-write file\n";
    }

    void write(const std::string& data) override {
        std::cout << "Writing to read-write file: " << data << "\n";
    }
};
```



How did Barcelona lose????

```
class ISoccerPlayer {
public:
    virtual void passBall() = 0;
    virtual void shoot() = 0;
    virtual void tackle() = 0;
    virtual void keepGoal() = 0;
    virtual ~ISoccerPlayer() = default;
};

class Striker : public ISoccerPlayer {
public:
    void passBall() override { std::cout << "Striker passes.\n"; }
    void shoot() override { std::cout << "Striker shoots!\n"; }
    void tackle() override { std::cout << "Striker tackles.\n"; }
    void keepGoal() override {
        throw std::logic_error("Striker can't keep goal!");
    }
};
```


That's the way to do it!

```
class IFieldPlayer {
public:
    virtual void passBall() = 0;
    virtual void shoot() = 0;
    virtual void tackle() = 0;
    virtual ~IFieldPlayer() = default;
};

class IGoalkeeper {
public:
    virtual void keepGoal() = 0;
    virtual void distributeBall() = 0;
    virtual ~IGoalkeeper() = default;
};

class Striker : public IFieldPlayer {
public:
    void passBall() override { std::cout << "Striker passes.\n"; }
    void shoot() override { std::cout << "Striker shoots!\n"; }
    void tackle() override { std::cout << "Striker tackles.\n"; }
};

class Goalkeeper : public IGoalkeeper, public IFieldPlayer {
public:
    void keepGoal() override { std::cout << "Goalkeeper saves!\n"; }
    void distributeBall() override { std::cout << "Goalkeeper passes.\n"; }
    void passBall() override { std::cout << "Goalkeeper passes from the back.\n"; }
    void shoot() override { std::cout << "Goalkeeper clears the ball.\n"; }
    void tackle() override { std::cout << "Goalkeeper dives for the ball.\n"; }
};
```

I know that the spades are the sword of a soldier...

```
class Sword {  
public:  
    void attack() {  
        std::cout << "Swinging sword!\n";  
    }  
};
```

```
class Warrior {  
    Sword sword;  
public:  
    void fight() {  
        sword.attack();  
    }  
};
```

That's the way to do it!

```
class IWeapon {
public:
    virtual void attack() = 0;
    virtual ~IWeapon() = default;
};

class Sword : public IWeapon {
public:
    void attack() override {
        std::cout << "Swinging sword!\n";
    }
};

class Bow : public IWeapon {
public:
    void attack() override {
        std::cout << "Shooting arrow!\n";
    }
};

class Warrior {
    IWeapon* weapon;
public:
    Warrior(IWeapon* w) : weapon(w) {}

    void fight() {
        weapon->attack();
    }
};
```



Am I right (or left) wing??



```
class Politician {
public:
    void draftLaw(const std::string& topic) {
        std::cout << "Writing a law about " << topic << std::endl;
    }

    void implementLaw() {
        std::cout << "Also implementing the law I wrote...\n";
    }

    void investigateCorruption() {
        std::cout << "Investigating myself... looks clean to me!\n";
    }

    void tweet() {
        std::cout << "Tweeting something controversial!\n";
    }

    void approveBudget() {
        if (rand() % 2 == 0)
            std::cout << "Approving budget.\n";
        else
            std::cout << "Blocking budget for attention.\n";
    }
};
```

That's the way to do it!

```
class ILegislator {
public:
    virtual void draftLaw(const std::string&) = 0;
    virtual ~ILegislator() = default;
};

class IExecutive {
public:
    virtual void implementLaw() = 0;
    virtual ~IExecutive() = default;
};

class IOversight {
public:
    virtual void investigate() = 0;
    virtual ~IOversight() = default;
};

class ISpokesperson {
public:
    virtual void tweet() = 0;
    virtual ~ISpokesperson() = default;
};

class IBudgetStrategy {
public:
    virtual void approveBudget() = 0;
    virtual ~IBudgetStrategy() = default;
};
```

That's the way to do it! - cont.

Senator → implements ILegislator

President → IExecutive, IBudgetStrategy

AuditorGeneral → IOversight

PressSecretary → ISpokesperson